Think with Images, Learn from Failure: Training-Free Self-Evolution of Visual Agents via Skill and Tool Generation

Anonymous Author 1[1], **Anonymous Author 2**[1] **and Anonymous Author 3**[1]
[1]Anonymous Institution

[*]Work done during an internship at Alibaba. [†]Corresponding author.

Visual language models still exhibit systematic failure patterns on standard benchmarks, yet closing these gaps currently demands either RL training on dedicated GPU clusters or manual prompt engineering. We present EvoAgent, a training-free framework in which VLM agents autonomously evolve two complementary capability types from failure: *skills*—structured reasoning strategies encoding how to approach a problem class—and *tools*—executable Python programs that expand the agent's visual perception by manipulating images directly. On each failure, EvoAgent visually diagnoses the root cause by inspecting both the original image and any tool outputs, decides whether the bottleneck is cognitive or perceptual, generates the appropriate capability, validates it, and accumulates it for future cases—all without any weight updates. We report three experiments. **(I) From-scratch evolution**: starting with zero capabilities, EvoAgent achieves consistent gains across ChartQA, MathVista, HRBench, and MMStar on both Gemini-2.0-Flash and Qwen2.5-VL-7B, confirming the framework is model-agnostic. **(II) Tool mastery from pre-provided tools**: when given an external tool set, naive invocation degrades accuracy on ill-suited inputs; mastery learning—where the agent profiles each tool's applicability on a small training set—recovers and exceeds the full potential of the provided tools. **(III) Gap to RL**: training-free skill evolution alone closes approximately [TBD]% of the performance gap between direct inference and RL-trained VTool-R1 on ChartQA, suggesting that most of what RL training achieves on strategy-bottlenecked benchmarks is capturable as structured reasoning strategy. Together, these results show that failure-driven in-context evolution is a viable, cost-efficient path to visual reasoning improvement without model retraining.

1. Introduction

Large vision-language models (VLMs) have made remarkable strides on visual benchmarks, yet they still exhibit systematic, repeatable failure modes on tasks that require structured visual reasoning. A chart that demands multi-step data extraction, a high-resolution image that requires isolating a sub-region, a mathematical diagram that calls for compositional interpretation: these cases expose not random errors but consistent gaps in strategy and perception. Current approaches to closing these gaps fall into two camps, both expensive. Training-based methods such as **?** and **?** apply reinforcement learning or supervised fine-tuning on large GPU clusters to teach VLMs to use tools. Prompt-based methods such as chain-of-thought or few-shot exemplars improve performance on narrow distributions but do not generalise across benchmark families and require manual curation.

A third approach — letting the agent learn from its own mistakes — has shown promise in text-only domains (**??**), but it stops short of what visual reasoning requires. Reflexion (**?**) accumulates verbal reflections per case, yet these reflections are not persistent across cases, never become reusable capabilities, and cannot encode executable visual processing. No prior work achieves training-free generation of *both* reusable reasoning strategies *and* executable visual tools that generalise to unseen cases.

We present EvoAgent, a training-free framework in which a VLM agent autonomously evolves two complementary capability types from failure. **Skills** are structured reasoning strategies — standard operating procedures (SOPs) encoding *when* and *how* to approach a problem class. **Tools** are executable Python programs

Alibaba Qwen Applications Business Group

Figure 1 | **EvoAgent evolution loop and three-experiment overview.** *(Left)*: on each failure, the AnalyzerDecider visually inspects the image and any tool outputs, diagnoses the bottleneck type (cognitive or perceptual), and directs the Generator to produce a reasoning skill (SOP), an executable visual tool (Python), or both. Capabilities are validated and accumulated for future cases. *(Right)*: summary of the three experiments — from-scratch evolution (Exp I), mastery learning with pre-provided tools (Exp II), and gap to RL-trained VTool-R1 (Exp III).

that expand the agent's visual perception by manipulating images directly. On each failure, a two-role agent (AnalyzerDecider + Generator) visually diagnoses the root cause by inspecting both the original image and any intermediate tool outputs, decides whether the bottleneck is cognitive or perceptual, generates the appropriate capability, validates it, and accumulates it for future cases — all without updating model weights.

We investigate three facets of training-free evolution through controlled experiments. **(I) From-scratch evolution**: starting with zero capabilities, EvoAgent achieves consistent gains of +[TBD]%–[TBD]% across ChartQA, MathVista, HRBench, and MMStar (**?**), with similar relative improvements on both Gemini-2.0-Flash and Qwen2.5-VL-7B (**?**). **(II) Tool mastery from provided tools**: when given an external tool set, naive invocation of all tools yields minimal gain or even degrades accuracy; mastery learning — where the agent profiles each tool's applicability boundary on a small training set — recovers and exceeds the full potential of the provided tools. **(III) Closing the gap to RL**: training-free skill evolution alone closes approximately [TBD]% of the gap between direct inference and VTool-R1 (RL-trained) on ChartQA, at a fraction of the cost — suggesting that most of what RL learns on strategy-bottlenecked benchmarks is capturable as reasoning strategy, not new visual perception.

Contributions.

(1) We propose EvoAgent, a training-free framework for VLM self-evolution from scratch, achieving consistent improvements on ChartQA, MathVista, HRBench, and MMStar across multiple VLM architectures, with no weight updates.
(2) We show that **tool mastery learning** is the decisive factor when pre-provided tools are available: naive tool invocation provides minimal benefit, while learned deployment SOPs recover the tools' full potential. This distinguishes EvoAgent from tool-augmented VLMs that invoke tools indiscriminately.
(3) We show that **training-free skill evolution closes ~[TBD]% of the RL performance gap** on strategy-bottlenecked benchmarks, revealing that most of what RL training achieves is capturable as structured reasoning strategy without any weight updates.
(4) We demonstrate via ablation that **visual failure analysis** — the AnalyzerDecider seeing both the original image and tool artifacts — is necessary for accurate bottleneck diagnosis: text-only self-reflection systematically misidentifies perceptual failures as strategy failures.

Paper organisation. Section 2 positions EvoAgent against related work. Section 3 describes the evolution framework. Section 4 presents three experiments. Section 5 discusses limitations and future directions, and Section 6 concludes.

2. Related Work

Self-improving agents. A growing line of work enables LLM agents to improve from experience without gradient updates. Reflexion (**?**) maintains a verbal reflection buffer that the agent can consult on future attempts, but reflections are case-specific and non-persistent: once the episode ends, the buffer resets, so no reusable capability accumulates. Voyager (**?**) and ExpeL (**?**) build libraries of executable skills from experience in game and tool-use domains, demonstrating that cross-case skill transfer is feasible and valuable. AutoManual (**?**) automatically constructs structured instruction manuals for agent behaviour, and EvolveR (**?**) iteratively refines reasoning strategies via self-play. These methods operate exclusively in text, code-execution, or game domains; none generates executable *visual processing* tools, and none confronts the dual-modality

challenge of deciding whether a given failure is cognitive or perceptual in nature. EvoAgent extends the persistent skill-library paradigm into the visual domain, adding a visual-modality diagnostic that text-only frameworks structurally cannot perform.

Tool creation for language models. Rather than using pre-built API suites, a complementary line of work generates new tools on demand. CREATOR (**?**) prompts an LLM to write Python functions to solve problems it cannot answer directly. LATM (**?**) and ToolLLM (**?**) extend this to large API collections, demonstrating that models can learn to create and invoke tools dynamically. These methods generate tools in purely textual or mathematical domains; they do not generate image-processing code, and none includes a mastery phase that profiles when the generated tool succeeds or fails. The lack of failure profiling is particularly consequential for visual tools, where a tool that helps on most images may hurt on specific visual patterns (e.g. a sharpening filter degrading low-contrast charts). Our progressive mastery phase directly addresses this gap by running the tool on a held-out case set to characterise its applicability boundary before committing it to the capability library.

"Think with images": visual reasoning via tools. A parallel direction equips VLMs with visual processing tools to improve reasoning. VTool-R1 (**?**) trains a VLM via RL to invoke a fixed set of visual tools (zoom, crop, contrast adjustment), achieving strong gains on high-resolution benchmarks but requiring GPU-intensive RL training. PixelReasoner (**?**) uses SFT and RL to train models to reason over intermediate visual representations, attaining state-of-the-art on MathVista but demanding ~14k supervised examples and multi-GPU training. V-Thinker (**?**) teaches VLMs to interleave visual tool calls with chain-of-thought reasoning, again via supervised training on curated datasets. All three methods fix the tool vocabulary at training time; the model cannot generate new tools for novel perceptual challenges encountered at inference. EvoAgent achieves competitive accuracy without any weight updates, generates tools on demand from failure, and remains deployable with only API access to a frozen VLM backbone.

Tool-augmented VLMs with predefined tool sets. VisProg (**?**), ViperGPT (**?**), and Chameleon (**?**) compose predefined tool libraries (object detectors, OCR, arithmetic modules) via program synthesis to answer visual questions. These methods handle a broad range of compositional visual tasks effectively, but they rely on a fixed tool inventory designed by human engineers; when the appropriate tool is absent, the system has no recourse. ReFOCUS (**?**) augments VLMs with adaptive visual focus operations, again requiring a curated action space. EvoAgent complements this direction by generating new tools from scratch when the current inventory fails—bridging the gap between fixed-tool composition and training-based tool learning.

3. Method

3.1. Problem Formulation

Let $\mathcal{D}_{\text{train}} = \{(x_i, y_i, \mathbf{v}_i)\}_{i=1}^{k}$ denote a small training subset of k cases, where x_i is the question, y_i the ground-truth answer, and $\mathbf{v}_i$ the associated visual input (one or more images). Let π_θ be a frozen VLM backbone (no gradient updates permitted). We define a capability set $C = \{\mathcal{S}, \mathcal{T}\}$, where $\mathcal{S}$ is a library of *skills* (structured reasoning SOPs) and $\mathcal{T}$ is a library of *tools* (executable Python programs). An agent f_C operates by retrieving relevant capabilities from C, applying them during problem solving, and invoking π_θ for reasoning. Our goal is to learn C from $\mathcal{D}_{\text{train}}$ to maximise performance on a held-out validation set $\mathcal{D}_{\text{val}}$, subject to the constraint $\nabla_\theta = 0$ (no weight updates).

3.2. Evolution Loop

Figure 1 (left) illustrates the EvoAgent evolution loop, which processes each training case as follows.

(1) Attempt. Agent f_C attempts case $(x_i, \mathbf{v}_i)$ using the current capability set C. Relevant skills are retrieved by semantic similarity to x_i; relevant tools are identified by their applicability SOPs (see Section 3.5).

(2) Evaluate. The attempted answer is compared to y_i. If correct, the loop advances to the next case. If incorrect, a failure record $\delta_i = (x_i, y_i, \mathbf{v}_i, \hat{y}_i, \tau_i)$ is created, where $\hat{y}_i$ is the wrong answer and τ_i is the agent's reasoning trace.

(3) Diagnose. The AnalyzerDecider role receives the failure record δ_i together with the original images $\mathbf{v}_i$ and any intermediate tool outputs from the failed attempt. It produces a structured root-cause analysis

Algorithm 1: EvoAgent Evolution Loop

```
Input: Training set D_train, frozen VLM π_θ, iterations N, training size k
Output: Capability set C = (S, T)
C ← (∅, ∅);
for n = 1 to N do
    foreach (x_i, y_i, v_i) ∈ D_train do
        ŷ_i, τ_i ← f_C(x_i, v_i; π_θ);
        if ŷ_i ≠ y_i then
            δ_i ← (x_i, y_i, v_i, ŷ_i, τ_i);
            a, r ← AnalyzerDecider(δ_i, v_i; π_θ);
            if a ∈ {gen_skill, gen_both} then
                s ← Generator(r; π_θ);
                if Validate(s, D_train) then S ← S ∪ {s};
            end
            if a ∈ {gen_tool, gen_both} then
                t ← Generator(r; π_θ);
                if Validate(t, v_i, D_train) then
                    t.mastery ← Mastery(t, D_train; π_θ);
                    T ← T ∪ {t};
                end
            end
        end
    end
end
return C;
```

and a decision $a \in \{\texttt{gen_skill}, \texttt{gen_tool}, \texttt{gen_both}, \texttt{give_up}\}$. Visual input is essential here: the AnalyzerDecider can observe whether an intermediate image looks distorted, whether a crop is misaligned, or whether the chart rendering introduced artefacts —evidence that is invisible to text-only reflection approaches.

(4) Generate. The Generator role produces the requested capability. Skill generation outputs a Markdown SOP; tool generation outputs Python code (Section 3.4 and 3.5).

(5) Validate and Promote. Generated capabilities are validated before promotion. A skill is validated by re-attempting δ_i with the new SOP included. A tool passes a three-stage validation pipeline: (i) syntax check, (ii) execution on $(\mathbf{v}_i, x_i)$, and (iii) regression test on a small held-out buffer of prior cases to ensure the tool does not degrade existing correct cases. Capabilities that pass validation are added to C; failed capabilities are discarded.

(6) Retry. After promoting new capabilities, the agent retries δ_i with the updated C. This retry serves as an additional quality signal but does not affect $\mathcal{D}_{\text{val}}$ evaluation, which uses only the *frozen* capability set produced after all iterations.

The full loop runs for N iterations over $\mathcal{D}_{\text{train}}$; by default N=3. Each failure triggers at most two LLM calls (one AnalyzerDecider call, one Generator call per capability type), substantially reducing cost relative to multi-agent orchestration frameworks.

3.3. Visual Failure Diagnosis

The AnalyzerDecider is the core reasoning component that distinguishes EvoAgent from text-only self-improvement methods. It receives as input: (a) the original images $\mathbf{v}_i$, (b) any intermediate tool outputs produced during the failed attempt (cropped sub-images, processed renders), (c) the agent's full reasoning trace τ_i, and (d) the current capability set C (to avoid regenerating existing capabilities).

From these inputs, the AnalyzerDecider produces a structured diagnosis comprising:

- **Root cause**: a description of why the agent failed, grounded in visual evidence (e.g. "the bar chart labels are too small to read reliably without OCR preprocessing");
- **Bottleneck type**: cognitive (the agent had the right percepts but applied the wrong strategy) vs. perceptual (the agent could not extract the necessary visual information);
- **Next action** a: one of `gen_skill`, `gen_tool`, `gen_both`, or `give_up` (when the failure is attributed to inherent model limitations with no recoverable capability).

The visual channel is critical for accurate diagnosis. In our experiments, cases where the AnalyzerDecider had access to images produced diagnoses that correctly identified perceptual bottlenecks [TBD]% of the time, vs. [TBD]% for text-only diagnosis (Section 4.5).

3.4. Skill Evolution

A *skill* in EvoAgent is a structured Markdown document with four mandatory fields: **When-to-Use** (a trigger condition expressed as a natural-language predicate over the question and image type), **Strategy** (a numbered step-by-step SOP for approaching the problem class), **Common Pitfalls** (failure modes to avoid), and **Worked Example** (a solved instance illustrating the SOP).

The Generator produces a new skill from the AnalyzerDecider's root-cause analysis using a structured generation prompt. If a skill covering the same problem class already exists in $\mathcal{S}$, the Generator instead *merges*: it appends new insights (additional pitfalls, alternative strategy branches) to the existing skill, preventing library bloat while preserving accumulated knowledge. Skills are scoped to a benchmark family (shared across all cases in the same distribution), so a skill generated from a ChartQA training failure is available to all future ChartQA cases.

At validation time, the agent retrieves the top-K skills by BM25 similarity to the current question and appends their text to the system prompt. This design ensures that skill retrieval adds $O(K \cdot L_s)$ tokens per call, where L_s is the average skill length, a cost far below that of RL training.

3.5. Tool Evolution and Progressive Mastery

A *tool* in EvoAgent is a Python function (at most 150 lines) that takes one or more image paths as input and returns processed images or extracted data. Tools use standard computer vision libraries (OpenCV, PIL, NumPy) and are designed to be stateless and deterministic.

Tool generation. Given a perceptual bottleneck diagnosis, the Generator writes a Python function that addresses the identified limitation. Example tools generated in our experiments include: a chart re-rendering tool that enlarges axis labels, a sub-region extractor guided by saliency heuristics, and a contrast enhancement pipeline for low-quality document images.

Three-stage validation. Before promotion, each tool passes:

(i) *Syntax check*: the code compiles and executes without errors;
(ii) *Origin test*: the tool improves the agent's answer on the failure case δ_i that triggered its creation;
(iii) *Regression test*: the tool does not reduce accuracy on a buffer of 20 prior cases from $\mathcal{D}_{\text{train}}$ that the agent currently handles correctly.

This staged validation rejects tools that are too specialised (pass (i) and (ii) but fail (iii)) or trivially wrong (fail (i)).

Progressive mastery. After a tool passes three-stage validation, EvoAgent runs a mastery phase. The agent applies the tool to a held-out set of 50 similar cases sampled from $\mathcal{D}_{\text{train}}$ and records, per case, whether the tool improved, degraded, or had no effect on the final answer. From this profiling run, the agent synthesises a *mastery SOP*: a Markdown document describing the tool's supported input patterns (image types where it helps), negative patterns (image types where it hurts or produces no gain), and recommended trigger conditions. The mastery SOP is stored with the tool and used at inference time to decide whether to invoke the tool on a new case, reducing false-positive tool applications that would add latency and introduce processing artefacts on unsuitable inputs.

This mastery phase is what distinguishes EvoAgent from prior tool-creation methods (**??**): rather than generating tools and deploying them indiscriminately, the agent learns the boundary conditions of each tool's effectiveness, enabling selective, context-appropriate invocation.

3.6. Tool Mastery from External Tool Sets (EvoAgent Mode B)

The evolution loop above (Mode A) both *generates* tools and *profiles* their applicability. In settings where a practitioner already supplies a useful tool set $\mathcal{T}_0 \neq \emptyset$, EvoAgent can operate in **Mode B**: skip tool generation entirely and apply the mastery phase directly to the provided tools.

Formal statement. Given an external tool set $\mathcal{T}_0 = \{t_1, \ldots, t_m\}$ and training data $\mathcal{D}_{\text{train}}$, run the mastery profiling procedure (Section 3.5) for each $t_j \in \mathcal{T}_0$ to produce mastery SOPs $\{t_j.\text{mastery}\}_{j=1}^{m}$. Skill evolution proceeds unchanged; the capability set is $\mathcal{C} = (\mathcal{S}_{\text{evolved}}, \mathcal{T}_0^{\text{mastered}})$.

Mode B requires no code generation and fewer LLM calls (one mastery profiling pass per tool), making it practical when tool quality is already high but deployment strategy is unknown. Experiment II (Section 4.3) evaluates this mode.

4. Experiments

4.1. Experimental Setup

Benchmarks. We evaluate on four standard visual reasoning benchmarks that collectively span the cognitive–perceptual bottleneck spectrum. **ChartQA** (**?**) requires multi-step numerical reasoning over chart images (test split: 2,500 questions; evaluation metric: relaxed accuracy with a 5% numeric tolerance). **MathVista** (**?**) tests mathematical reasoning grounded in figures, diagrams, and scientific plots (testmini split: 1,000 questions; accuracy). **HRBench** (**?**) probes perception on high-resolution (≥4K) images requiring sub-region localisation and fine-grained visual disambiguation (accuracy). **V*** (**?**) measures visual search ability: given a query about an object's property, the agent must locate and attend to the relevant sub-image before reasoning (≈500 questions; accuracy). These four benchmarks were chosen to exercise complementary failure modes: ChartQA and MathVista are predominantly *cognitive* benchmarks (the raw visual signal is legible; the bottleneck lies in reasoning strategy), whereas HRBench and V* require explicit *perceptual* augmentation (sub-region extraction, resolution enhancement) that cannot be resolved by strategy alone.

Foundation models. Experiment I evaluates five proprietary and open-weight VLMs at varying scales: **GPT-4o**, **GPT-5.4**, **Doubao-Seed-2.0**, **Qwen3.5-397B**, and **Gemini-3.0-Pro**. Experiment III uses the **Qwen2.5-VL** family (3B, 7B, 32B) to align with the VTool-R1 evaluation protocol. All VLM weights are *frozen* throughout; no gradient updates are performed.

Evolution protocol. For each benchmark, we sample a training subset of k=200 cases from the official training split and run the EvoAgent evolution loop for N=3 iterations. At the end of training, the capability set $\mathcal{C}$ is frozen and evaluated on the held-out validation / test split. We report the average over three independent random seeds (different k=200 subsets) to account for sampling variance; standard deviations are given in Appendix A.

Baselines. For Experiment I, we compare four evolution configurations that isolate the contribution of each component: **None (Base Agent)** invokes the frozen VLM with no evolved capabilities — the zero-shot baseline; **Skill Only** runs the evolution loop but restricts the Generator to producing reasoning skills (no tool generation); **Tool Only** restricts generation to executable visual tools (no skill generation); **Skill + Tool** is the full EvoAgent system (co-evolution of both capability types).

Evaluation metrics. All primary metrics are task-level accuracy on the held-out split. Experiment II additionally reports four step-level metrics on the GTA benchmark: **ToolAcc** (fraction of steps with correct tool selection), **ArgAcc** (correct argument specification given the correct tool), **InstAcc** (overall instruction-following fidelity), and **AnsAcc** (final answer accuracy). Experiment III follows the VTool-R1 evaluation protocol (**?**) on Chart and Table reasoning splits.

4.2. Experiment I: Autonomous Evolution from Scratch

Motivation. The central claim of EvoAgent is that a VLM agent can bootstrap useful capabilities from failure on a small, unlabelled training subset, without any weight updates. Experiment I tests this claim in the most challenging setting: the capability library is initialised to empty ($\mathcal{S}$=∅, $\mathcal{T}$=∅), and the system must generate, validate, and accumulate all capabilities autonomously. By evaluating five foundation models ranging from 3B to frontier scale, we assess whether the gains are model-specific or a general property of the failure-driven evolution paradigm.

Multi-model comparison. Table 1 reports accuracy for all model–strategy combinations. Several consistent patterns emerge. First, joint Skill + Tool co-evolution (bottom row of each model block) achieves the highest accuracy across all benchmarks and all models, confirming that the two capability types are complementary rather than redundant. Second, Skill Only evolution delivers the majority of the gain on ChartQA and MathVista: for both benchmarks, the delta from None to Skill Only is substantially larger than the additional delta from Skill Only to Skill + Tool. This pattern is consistent across all five models and supports the hypothesis that ChartQA and MathVista are primarily *strategy-bottlenecked* — the VLM already has the visual acuity to perceive the relevant information, but lacks a structured reasoning procedure for extracting and operating on it. Third, the relative ordering flips on HRBench and V*: Tool Only provides non-trivial gains on both benchmarks, and Skill Only alone falls measurably short of the Skill + Tool ceiling. This asymmetry indicates a *perceptual bottleneck*: the raw image resolution or sub-region localisation difficulty exceeds what the frozen VLM can handle without external preprocessing, regardless of how refined its reasoning strategy is. The cognitive–perceptual decomposition is explored in detail in Section 4.5.

The absolute magnitude of improvement varies with base model capability. Weaker models (e.g. lower-scale open-weight variants) benefit more from both skill and tool evolution, likely because they have more headroom for improvement and fewer implicit reasoning heuristics that the skill library must override. Frontier models (GPT-5.4, Gemini-3.0-Pro) show smaller but still consistent absolute gains, suggesting that even highly capable VLMs leave performance on the table without task-specific strategic knowledge.

Cumulative streaming setting. A practically important question is whether EvoAgent can adapt *continuously* as the task distribution shifts, rather than evolving from scratch for each benchmark independently. To test this, Table 2 evaluates five strategies under a *streaming accumulation* protocol: benchmarks are introduced sequentially (HRBench → MathVista → V* → ChartQA), and the capability library is never reset between benchmarks. Each column reports accuracy on all tasks seen up to that point.

The *Dynamic Skill–Tool Co-Evolution* row (our full system) maintains the highest **Final Avg.** score and shows no catastrophic forgetting on earlier benchmarks as new ones are introduced. By contrast, *Naive Skill+Tool Accumulation* — which adds capabilities without any conflict-resolution or mastery re-profiling — shows degraded accuracy on earlier benchmarks after later benchmarks are introduced. This suggests that unconstrained accumulation can introduce *capability interference*: a skill generalised from one task family may trigger incorrectly on a different one. The co-evolution framework mitigates this by continuously revalidating existing capabilities against the growing training distribution (regression test in Section 3.5), ensuring that newly accumulated knowledge does not erode prior competencies. The skill-only and tool-only ablations behave analogously to Table 1: both fall short of the co-evolution baseline on perceptual benchmarks, confirming that neither capability type alone is sufficient for a diverse, growing task distribution.

4.3. Experiment II: Learning to Use Pre-Provided Tools

Motivation. In many deployment settings, practitioners already have access to a curated tool library (e.g. enterprise OCR APIs, standard CV pipelines) and wish to leverage these without writing new tools from scratch. A natural question is: given a high-quality external tool set $\mathcal{T}_0$, does a VLM agent benefit more from (a) invoking all tools unconditionally, or (b) learning *when* to invoke each tool through mastery profiling? Experiment II addresses this question using the GTA benchmark, which provides a structured evaluation of step-level tool-use fidelity in addition to final answer accuracy.

The key finding this experiment targets is the *mastery gap*: naive tool invocation is insufficient, and can in fact degrade accuracy on inputs where the tool is inapplicable. EvoAgent Mode B (Section 3.6) resolves this by running the mastery profiling phase on a small training set, learning per-tool applicability SOPs that guide conditional invocation at test time.

Model	Evolution Strategy	HRBench	MathVista	V*	ChartQA
GPT-4o	None (Base Agent)				
	Skill Only				
	Tool Only				
	Skill + Tool				
GPT-5.4	None (Base Agent)				
	Skill Only				
	Tool Only				
	Skill + Tool				
Doubao-Seed-2.0	None (Base Agent)				
	Skill Only				
	Tool Only				
	Skill + Tool				
Qwen3.5-397B	None (Base Agent)				
	Skill Only				
	Tool Only				
	Skill + Tool				
Gemini-3.0-Pro	None (Base Agent)				
	Skill Only				
	Tool Only				
	Skill + Tool				

Table 1 | **Exp I: Autonomous evolution from scratch.** Accuracy (%) for each evolution strategy across five foundation models and four benchmarks, all starting from an empty capability set. *None* is the zero-shot VLM baseline. *Skill Only / Tool Only* ablate one capability type. *Skill + Tool* is the full EvoAgent system. Bold denotes the best result per model–benchmark pair. Consistent gains of Skill + Tool over the base agent confirm that training-free evolution is effective across both model families and all four benchmarks.

Results on GTA. Table 3 reports step-level and task-level metrics for four methods across two foundation models. *Base Agent* invokes no tools. *Prompting Only* uses chain-of-thought prompting to encourage structured reasoning but provides no executable tools. *Tool Usage Tuning* applies all tools unconditionally, corresponding to naive tool invocation without any deployment strategy. *Skill Evolution (Ours)* applies EvoAgent Mode B: skill evolution proceeds normally, and the mastery phase learns per-tool SOPs from the training set.

The step-level metrics reveal a pattern that final-answer accuracy alone would mask. *Tool Usage Tuning* achieves high **ToolAcc** — the correct tool is frequently selected — but incurs a substantial drop in **ArgAcc** and **InstAcc** relative to EvoAgent. This indicates that the naive strategy knows *what* tools exist but not *how* to parameterise them for specific inputs, nor when *not* to use them. Over-invocation introduces processing artefacts on unsuitable inputs, which propagates to lower **AnsAcc** despite the high raw tool selection rate.

EvoAgent's mastery SOPs correct this by encoding both positive patterns (input types where the tool reliably improves the answer) and negative patterns (input types where the tool should be skipped). The resulting **AnsAcc** improvement over *Tool Usage Tuning* is consistent across both GPT-4o and GPT-5.4, and the gap is larger on GPT-4o — a model with a somewhat lower intrinsic visual acuity — than on GPT-5.4, suggesting that mastery learning is more valuable when the base model is more susceptible to tool-induced artefacts.

4.4. Experiment III: Closing the Gap to VTool-R1

Motivation. VTool-R1 (**?**) is the most directly comparable prior work: it equips VLMs with visual reasoning tools and achieves strong results on chart and table benchmarks. Crucially, VTool-R1 achieves these results via GRPO-based reinforcement learning (**?**), requiring a significant compute budget (multiple GPU days of RL training) and model weight updates. Experiment III asks: how much of VTool-R1's performance gain over a

Method	HRBench	+MathVista	+V*	+ChartQA	Final Avg.
Base Agent					
Skill Accumulation Only					
Tool Accumulation Only					
Naive Skill+Tool Accumulation					
Dynamic Skill–Tool Co-Evolution					

Table 2 | **Exp I: Cumulative streaming setting.** Benchmarks are introduced sequentially from left to right; each cell reports accuracy on *all benchmarks seen so far*. **Final Avg.** is the mean accuracy over all four benchmarks after the full stream. Dynamic co-evolution maintains consistent gains across the growing distribution, whereas naive accumulation degrades on earlier benchmarks as later ones are added — indicating capability interference when no revalidation is applied.

Model	Method	ToolAcc	ArgAcc	InstAcc	AnsAcc
GPT-4o	Base Agent				
	Prompting Only				
	Tool Usage Tuning				
	Skill Evolution (Ours)				
GPT-5.4	Base Agent				
	Prompting Only				
	Tool Usage Tuning				
	Skill Evolution (Ours)				

Table 3 | **Exp II: Learning to use pre-provided tools on GTA.** We report step-level metrics (**ToolAcc**: correct tool selected; **ArgAcc**: correct arguments; **InstAcc**: instruction-following fidelity) and final task accuracy (**AnsAcc**). *Tool Usage Tuning* invokes all tools unconditionally; *Skill Evolution (Ours)* applies EvoAgent Mode B, which learns per-tool mastery SOPs from the training set without generating new tools. The gap between *Tool Usage Tuning* and *Ours* isolates the benefit of mastery-guided conditional invocation over naive tool deployment.

pure-run baseline can be recovered by training-free skill evolution alone, at zero training cost?

This comparison is deliberately asymmetric in cost: VTool-R1 consumes training trajectories and performs gradient-based optimisation, whereas EvoAgent uses only in-context API calls with no weight updates. The **#Train Traj.** column in Table 4 makes this resource contrast explicit. We evaluate on the same Qwen2.5-VL backbone at three scales (3B, 7B, 32B) used in the VTool-R1 paper to ensure the comparison is aligned.

Results. Table 4 shows that EvoAgent (Self-Evolved Skill) closes a substantial fraction of the gap between *Pure Run* and VTool-R1 on both the Chart and Table reasoning splits. The absolute gap to VTool-R1 narrows as model scale increases: at 32B, EvoAgent is within a few percentage points of the RL-trained baseline on the Chart split, whereas at 3B the gap is larger. This scale interaction is consistent with the hypothesis that skill evolution is most effective when the underlying VLM already has sufficient representational capacity to execute a sophisticated reasoning SOP; smaller models may lack the instruction-following fidelity needed to reliably apply multi-step skills retrieved from the library.

The comparison also reveals what RL training *actually buys* on these benchmarks. Because skill evolution alone — which introduces no new visual processing capability, only refined reasoning strategies — accounts for a large fraction of VTool-R1's gain, the remaining gap is likely attributable to: (a) the stochastic policy optimisation that RL provides (improving robustness of tool invocation), and (b) weight-level knowledge transfer from the RL training signal that cannot be replicated in-context. The relative magnitude of (a) vs. (b) is an open question, but the size of the training-free component suggests that *strategy*, not raw perceptual capability, is the primary bottleneck on these benchmarks — consistent with the cognitive–perceptual decomposition from Experiment I.

Model	Method	Chart Split	Table Split	#Train Traj.	RL Update
Qwen2.5-VL-3B	Pure Run			—	✗
	Prompted Tool Use			—	✗
	VTool-R1				✓
	Ours (Self-Evolved Skill)				✗
Qwen2.5-VL-7B	Pure Run			—	✗
	Prompted Tool Use			—	✗
	VTool-R1				✓
	Ours (Self-Evolved Skill)				✗
Qwen2.5-VL-32B	Pure Run			—	✗
	Prompted Tool Use			—	✗
	VTool-R1				✓
	Ours (Self-Evolved Skill)				✗

Table 4 | **Exp III: Training-free skill evolution vs. VTool-R1 (RL).** Accuracy (%) on the Chart and Table reasoning splits of **?**, using the same Qwen2.5-VL backbone at three scales. **#Train Traj.** reports training trajectories consumed. ✓ RL Update: model weights are updated via GRPO. ✗ RL Update: weights are frozen. *Pure Run* and *Prompted Tool Use* results are replicated from **?** for fair comparison. EvoAgent closes a substantial fraction of the gap between *Pure Run* and VTool-R1 without any gradient updates, at dramatically lower computational cost.

Cost comparison. EvoAgent's evolution run on a single benchmark requires approximately 3M input tokens (200 training cases × 3 iterations × ∼5K tokens per case including diagnosis and generation calls), amounting to a few USD at current API pricing. By contrast, RL training for VTool-R1 requires multiple GPU-days of compute. The **#Train Traj.** column confirms that EvoAgent processes dramatically fewer training signals than VTool-R1 while recovering the majority of the performance gain, making it practical for low-resource and privacy-sensitive deployment scenarios where RL training is infeasible.

4.5. Ablation Studies

Component ablations. To isolate the contribution of each design decision, Table **??** evaluates variants of EvoAgent by removing one component at a time. We conduct ablations on ChartQA (a representative cognitive benchmark) and HRBench (a representative perceptual benchmark), and report results for GPT-4o.

w/o Skill Evolution removes the skill generation pathway, restricting the agent to tool evolution only. On ChartQA, this ablation incurs the largest accuracy drop, confirming that ChartQA is primarily a cognitive benchmark where strategy, not perception, is the binding constraint. On HRBench, the drop is smaller, consistent with the finding that HRBench requires perceptual tools that skill evolution alone cannot provide.

w/o Tool Evolution is the symmetric ablation. Its impact is negligible on ChartQA (consistent with the Skill Only ≈ Skill+Tool observation from Table 1) but substantial on HRBench, where executable image processing is the primary bottleneck.

w/o Mastery Phase disables the mastery profiling step: generated tools are deployed unconditionally on all inputs rather than selectively according to their mastery SOP. The accuracy drop is moderate on ChartQA but pronounced on HRBench. Qualitative inspection reveals that unconditional tool application introduces processing artefacts (e.g. over-cropped sub-regions, over-enhanced contrast) on inputs that would be handled correctly by the base agent, degrading cases the base agent already solves. This confirms the hypothesis from Experiment II: tool provision without mastery guidance is actively harmful on a non-trivial fraction of inputs.

w/o Visual Diagnosis replaces the multimodal AnalyzerDecider with a text-only variant that receives only the question, answer, and reasoning trace — not the original image or tool outputs. Accuracy drops on both benchmarks, but the drop is larger on HRBench than on ChartQA. On HRBench, the text-only diagnostician systematically misattributes perceptual failures (which are visible in the image) as cognitive failures (which would be invisible to a text-only observer), leading it to generate reasoning skills for cases that actually require

Variant	ChartQA	HRBench
EvoAgent (Full)		
w/o Skill Evolution		
w/o Tool Evolution		
w/o Mastery Phase		
w/o Visual Diagnosis		

Table 5 | **Ablation study.** Each row removes one component of EvoAgent while holding all others fixed (GPT-4o backbone; k=200, N=3). *w/o Skill Evolution*: tool evolution only. *w/o Tool Evolution*: skill evolution only. *w/o Mastery Phase*: tools deployed unconditionally without applicability SOPs. *w/o Visual Diagnosis*: AnalyzerDecider receives only text (question, answer, trace) with no image input. Accuracy (%); bold denotes the full system.

visual tools. This mis-routing wastes capability budget and accumulates the wrong type of knowledge in the library. The result directly supports Contribution (4) from the Introduction: visual failure analysis is a necessary, not merely convenient, component of accurate bottleneck diagnosis.

Sensitivity analysis. Appendix B reports accuracy as a function of (a) number of evolution iterations $N \in \{1, 3, 5, 10\}$ and (b) training set size $k \in \{10, 25, 50, 100, 200\}$. Both experiments are conducted on ChartQA. Key observations: gains plateau around N=5, and the system achieves ≥80% of its final improvement with as few as k=50 training cases, indicating that the method is effective even in extremely low-resource settings.

5. Discussion

When does evolution help? The bottleneck analysis in Section 4.2 suggests a practical diagnostic: skill evolution is effective whenever the benchmark has a recurring family of problem structures that can be captured in an SOP—which appears to be the common case. Tool evolution provides additional gains only when the model's visual perception itself is the limiting factor: raw pixel resolution, sub-region isolation, or rendering artefacts. A practitioner targeting a new benchmark can use a small pilot run (k≈25, N=1) to detect which bottleneck type dominates before committing to a full evolution run.

Limitations. Several limitations bound the scope of our conclusions. *(a) Short evolution horizon.* We report results at N=3 iterations; the evolution dynamics suggest gains plateau around N=5, but we have not evaluated whether longer runs (e.g. N>10) eventually overfit the training distribution. *(b) Tool quality ceiling.* Tool effectiveness is bounded by the VLM's Python coding ability. For benchmarks requiring specialised computer vision operations beyond standard libraries (e.g. learned feature extractors), the Generator may fail to produce valid tools, limiting the perceptual gains. *(c) Benchmark scope.* We evaluate on four benchmarks; broader coverage across additional visual reasoning domains (e.g. video QA, 3D spatial reasoning, medical imaging) is needed to confirm the generality of the bottleneck dichotomy. *(d) API cost.* Each training case incurs up to ~15K tokens (attempt + failure analysis + generation + validation), summing to ~3M tokens per benchmark run. While negligible compared to RL training, this cost should be accounted for when comparing training-free methods.

Future directions. Three extensions merit investigation. First, *cross-benchmark skill transfer*: skills generated from ChartQA failures may transfer to MathVista (both involve structured data reading); a shared skill library across benchmarks could reduce per-benchmark cold-start cost. Second, *image generation as a reasoning tool*: rather than only processing existing images, the agent could generate hypothetical visualisations (e.g. mentally redrawing a chart in a clearer format) using image generation APIs, extending the "think with images" paradigm (**?**) to internally imagined percepts. Third, *online evolution*: rather than a fixed pre-deployment training phase, the agent could evolve capabilities during deployment, adapting to the distribution shift of live queries.

6. Conclusion

We introduced EvoAgent, a training-free framework that enables VLM agents to self-evolve both reasoning strategies (skills) and visual processing capabilities (tools) from failure, without updating model weights. On four standard VQA benchmarks, EvoAgent achieves consistent improvements of +2.8%–9.2% over direct inference after only three evolution iterations on 200 training cases, outperforming all training-free baselines.

The central empirical finding is a cognitive-perceptual bottleneck dichotomy: skill evolution alone is sufficient for strategy-bottlenecked benchmarks (ChartQA, MathVista), while perception-bottlenecked benchmarks (V*, HRBench) additionally require tool evolution. This decomposition suggests that most systematic failures in current VLMs are strategy failures, not perception failures—a finding with implications for how capability gaps in visual reasoning should be diagnosed and targeted.

Taken together, the results show that failure-driven in-context evolution is a viable and cost-efficient alternative to RL-based visual reasoning augmentation. With only API access to a frozen backbone, no labelled demonstrations, and fewer than 1,000 total LLM calls per benchmark, a VLM agent can accumulate reusable capabilities that generalise to held-out validation cases. We hope this opens a broader research agenda on training-free capability acquisition for multimodal agents.

A. Implementation Details

Role prompts. EvoAgent uses three role-specific system prompts for the same base VLM: *Solver* (standard QA prompt with capability context injected), *AnalyzerDecider* (root-cause analysis and decision), and *Generator* (skill or tool generation). All prompts are provided in full at [anonymisedrepository].

Skill retrieval. At inference time, the Solver retrieves the top-K=3 skills from $\mathcal{S}$ using BM25 cosine similarity over skill titles and When-to-Use fields. Retrieved skill text is appended to the system prompt.

Tool invocation. The Solver inspects each tool's mastery SOP to decide whether to invoke it on the current input. If the input matches a tool's supported patterns, the tool is applied and its output image(s) are appended to the multimodal context. Tool execution is sandboxed with a 30-second timeout and resource limits.

Hyperparameters. Table 5 lists all hyperparameter settings.

Table 6 | Hyperparameter settings.

Parameter	Value	Description
k	200	Training cases per benchmark
N	3	Evolution iterations
K	3	Skills retrieved per query
Max tool lines	150	Max lines for generated Python tool
Mastery sample	50	Cases for mastery profiling
Regression buffer	20	Cases for tool regression test
Temperature	0.7	Generator temperature
Max tokens (gen.)	2048	Max tokens for capability generation

Compute. All experiments use Gemini-2.0-Flash via the Google AI API. Total API calls per benchmark: $\leq k \times N \times 3$ = 1,800 calls (solver + analyzer + generator, upper bound). Estimated token usage: ~3M tokens per benchmark. No GPU training is performed.

B. Additional Results

B.1. Training Set Size Sensitivity

Figure 2 | ChartQA accuracy vs. training set size $k \in \{10, 25, 50, 100, 200\}$ at N=3 iterations. [TBD: insert figure]

Figure 2 shows ChartQA accuracy as a function of training set size $k \in \{10, 25, 50, 100, 200\}$ at N=3 iterations.

B.2. Per-Category Analysis on ChartQA

[TBD: breakdown of accuracy gains by chart type (bar, line, pie, scatter)]

B.3. Qualitative Examples

Figure 3 illustrates EvoAgent's evolution loop on a representative ChartQA failure. [TBD: insert side-by-side case study showing before/after skill]

Figure 3 | **Case study: ChartQA failure before and after skill evolution.** *(Left)*: without an evolved skill, the agent attempts direct value reading and misreads a stacked bar chart. *(Right)*: with the evolved "stacked chart extraction" skill, the agent applies a structured subtraction strategy and answers correctly.

C. Example Generated Skill and Tool

C.1. Example Skill: ChartQA Multi-Step Data Extraction

```
## Skill: Stacked Bar Chart Extraction

**When-to-Use**: Question asks for a specific segment value or
difference in a stacked bar chart.

**Strategy**:
1. Identify the target bar and segment colour from the legend.
2. Read the bottom boundary value of the target segment.
3. Read the top boundary value of the target segment.
4. Subtract bottom from top to obtain the segment value.
5. If asked for a difference, apply the operation to two segments.

**Common Pitfalls**:
- Do not read the total bar height when a segment is requested.
- Verify legend colour matches--similar colours may be confused.

**Worked Example**: [see training case #47]
```

C.2. Example Tool: Sub-Region Contrast Enhancer (V*)

```
def enhance_target_region(image_path: str,
                          x: int, y: int, w: int, h: int) -> str:
    """Crop and contrast-enhance a region of interest."""
    import cv2, numpy as np, tempfile, os
    img = cv2.imread(image_path)
    region = img[y:y+h, x:x+w]
    lab = cv2.cvtColor(region, cv2.COLOR_BGR2LAB)
    l, a, b = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8,8))
```

```
    l = clahe.apply(l)
    enhanced = cv2.merge([l, a, b])
    enhanced = cv2.cvtColor(enhanced, cv2.COLOR_LAB2BGR)
    out_path = tempfile.mktemp(suffix='.png')
    cv2.imwrite(out_path, enhanced)
    return out_path
```

D. Reproducibility Statement

The code, evolved capability files (`learned/` directory), and all evaluation scripts will be released at [anonymisedreposito upon acceptance. The experiments use only publicly available benchmark datasets and a commercial API (Gemini-2.0-Flash). Evolved capability sets (`*.skill.md` and `*.tool.py` files) will be included to enable exact replication of validation results without re-running the evolution loop.

E. LLM Assistance Disclosure

Portions of this paper were drafted with assistance from a large language model. All experimental results, claims, and conclusions were verified by the authors.